Mécanismes de sécurité de la base de données - pili-pos

Conformément au §II.C du Manuel de certification SCE 2.0, ce document décrit les **mécanismes techniques et organisationnels** qui protègent la base de données fiscale contre la modification, la suppression et l'accès non autorisé.

1. Mécanismes au niveau SQLite

1.1 WAL (Write-Ahead Logging)

```
PRAGMA journal_mode = WAL;
```

Activé au démarrage par `pos_db::migrate::run_migrations` (`crates/pos-db/src/migrate.rs`, lignes 162 à 166 : `journal_mode=WAL`, `synchronous=FULL`, `foreign_keys=ON`).

Le mode WAL garantit : - **Atomicité crash-safe** : un événement fiscal n'est commité que lorsque toute la transaction (compteur + archive + clôture commande) a été inscrite dans le WAL et synchronisée sur disque. Le même démarrage fixe `PRAGMA synchronous = FULL` (test `commits_are_synchronous_full` dans `migrate.rs`) : chaque commit atteint le disque avant de retourner, y compris en mode WAL. La requête de signature est donc durable avant d'être envoyée au FDM. - **Cohérence en cas de coupure de courant** : SQLite rejoue ou abandonne automatiquement les transactions partielles au prochain démarrage. Aucun événement n'est jamais à moitié écrit.

1.2 Foreign keys

```
PRAGMA foreign_keys = ON;
```

Activé au démarrage. Empêche toute insertion `order_lines.order_id` ou `fiscal_events.order_id` qui pointerait sur une commande inexistante.

1.3 Contraintes CHECK

Au niveau colonne : - `fiscal_events.event_type IN ('N','P','T','C','I','F','S','R')` - rejette toute insertion avec un label invalide. - `orders.status IN ('open','closed','voided')` - rejette tout statut inconnu. - `orders.order_type IN ('dine_in','takeaway')` - rejette tout type inconnu. - `menu_items.vat_code IN ('A','B','C','D','X')` - rejette tout code TVA invalide. - `payments.method IN ('cash','card','meal_voucher','other')` (migration 8) - restreint les modes de paiement. - `users.role IN ('admin','cashier','waiter')` - restreint les rôles à ceux mappés sur les profils SCE 2.0. - `peppol_deliveries.status IN ('queued','accepted','failed')` - restreint les statuts de livraison Peppol.

1.4 Contraintes d'unicité

- `fiscal_events UNIQUE(event_type, sequence_number)` - empêche la double insertion du même couple (type, compteur). Couplée à la monotonie du compteur, garantit la **stricte séquentialité** exigée par la spec SCE 2.0.
- `users` primary key sur `insz` - un même INSZ ne peut pas être enregistré deux fois.
- `translations UNIQUE(entity_type, entity_id, field, lang)` - une seule traduction par couple entité / champ / langue.

2. Mécanismes au niveau applicatif

2.1 Append-only pour `fiscal_events`

Deux protections indépendantes, la seconde ne dépendant pas du code.

Triggers SQLite (niveau base). Le schéma refuse la modification et la suppression, même par un accès direct à la base :

- `fiscal_events_no_delete` et `fiscal_events_no_rewrite` (migration 4) : tout `DELETE` et tout `UPDATE` sur `fiscal_events` avorte (`fiscal events are append-only`) ;
- `z_reports_no_delete` et `z_reports_no_rewrite` (migration 4), même règle pour les rapports Z ;
- `fiscal_events_period_open` (migration 10) : un `INSERT` dont le `bookingPeriodId` désigne une période déjà close avorte ;
- `order_entries_no_delete` et `order_entries_no_rewrite` (migration 2), seul `sent_event_id` reste modifiable ;
- `booking_periods_no_reopen` et `booking_periods_no_delete` (migration 3) ;
- `pending_signs_request_immutable`, `pending_signs_context_on_resolution`, `pending_signs_no_reopen` et `pending_signs_no_delete` (migration 10) ;
- `ticket_deliveries_*`, `original_prints_*`, `order_price_changes_*` et `integration_switch_log_*` (liste complète dans `SCHEMA_ER_FR.md` §4).

Revue de code (niveau applicatif). Le code applicatif ne contient AUCUNE instruction `DELETE FROM fiscal_events` ni `UPDATE fiscal_events SET ...` (vérification grep - cf. `DECLARATION_FONCTIONS_INTERDITES_FR.md` §1 et §2). Les seules opérations possibles sont :

- `INSERT` via `pos_fiscal::archive::archive_event` (`crates/pos-fiscal/src/archive.rs:38`).
- `SELECT` via les fonctions de `pos_db::queries::fiscal`.

Tout patch introduisant un DELETE/UPDATE sur `fiscal_events` est rejeté en pull request (politique organisationnelle, cf. `PROFIL_ENTREPRISE_FR.md`). Les tests `archived_event_cannot_be_deleted` et `archived_event_cannot_be_rewritten` (`crates/pos-fiscal/src/archive.rs`) vérifient les triggers.

2.2 Transaction atomique compteur + archive

Chaque événement fiscal est signé sous un verrou unique, puis inséré dans une transaction SQLite qui inclut :

1. L'allocation du compteur (`FiscalCounter::next` - incrément persistant).
2. L'insertion de la ligne `fiscal_events`.
3. (Le cas échéant) la fermeture de la commande (`UPDATE orders SET status = 'closed'`).
4. La clôture de la ligne `pending_signs` de la requête (statut `signed`).

Si l'une de ces opérations échoue, **toute la transaction est annulée** (rollback) : ni le compteur ni l'archive ni le statut de commande ne sont modifiés, et la ligne `pending_signs` reste ouverte : l'opération suivante renvoie la requête à l'identique et archive l'événement (aucun événement signé n'est perdu). Le déclencheur `fiscal_events_period_open` refuse l'archivage dans une période déjà fermée. Voir `STRATEGIE_COMPTEURS_FR.md` §5 et la régression `pili-pos::spf_e2e_no_gap`.

2.3 Aucune suppression de compteur

Le code applicatif ne contient AUCUNE instruction `DELETE FROM fiscal_counters` ni `UPDATE fiscal_counters SET current_value = ...` ailleurs que dans `FiscalCounter::next` (qui incrémente exclusivement, jamais ne diminue). Voir `DECLARATION_FONCTIONS_INTERDITES_FR.md` §4.

2.4 Validation des entrées

- INSZ validé par `pos_core::user::validate_insz` (11 chiffres ASCII + mod-97) avant tout appel commande mutant un événement fiscal.
- Montants exprimés en **centimes en `i64`** - pas de flottants pour les calculs monétaires. Toute conversion depuis l'interface utilisateur passe par un parsing strict (panique si overflow ou caractères invalides).
- Horloge système **non modifiable** par l'opérateur (image kiosque, pas d'accès root, NTP via `chrony`).

3. Mécanismes au niveau système

3.1 Permissions de fichiers

Fichier	Propriétaire	Mode	Justification
`/var/lib/pili-pos/pos.db`	`pos:pos`	`0640`	Lecture/écriture par le service uniquement (le groupe `pos` donne un accès lecture au compte `spf`, voir §3.2)
`/var/lib/pili-pos/pos.db-wal`	`pos:pos`	`0640`	Idem
`/var/lib/pili-pos/pos.db-shm`	`pos:pos`	`0640`	Idem
`engine.lock` (dans `/var/lib/pili-pos/`)	`pos:pos`	`0640`	Fichier de verrou vide, sans aucune donnée : le moteur y prend un verrou exclusif à l'ouverture, pour qu'une seule instance écrive dans `pos.db`
`/var/lib/pili-pos/backups/`	`pos:pos`	`0750`	Répertoire des sauvegardes
`/var/lib/pili-pos/backups/pos-*.db`	`pos:pos`	`0640`	Sauvegardes journalières
`/var/lib/pili-pos/logs/pos.log.*`	`pos:pos`	`0640`	Journaux applicatifs
`/etc/pili-pos/pos.json`	`root:pos`	`0640`	Configuration, lecture par le service et le groupe `pos` uniquement
`/opt/pili-pos/pili-pos`	`root:root`	`0755`	Binaire, exécution publique

`/var/lib/pili-pos/` est le `StateDirectory=` de `pili-pos.service` ; systemd le crée avec `StateDirectoryMode=0750`. Le service tourne avec `UMask=0027`, donc tout fichier qu'il crée hérite directement des modes `0640`/`0750` ci-dessus sans étape de `chmod` séparée.

3.2 Utilisateur de service dédié

Le service `pili-pos.service` (systemd) tourne sous l'utilisateur non-root `pos`, créé lors de la construction de l'image (`deploy/provision/provision.sh --phase image`, appelé aussi bien pour l'image kiosque mkosi que pour une installation sur un Debian 13 déjà démarré). Il n'a PAS de shell de connexion (`/usr/sbin/nologin`) ni de mot de passe.

Extrait réel de `deploy/pili-pos.service` :

```
[Service]
Type=simple
User=pos
Group=pos
PAMName=pili-pos
TTYPath=/dev/tty1
ExecStart=/usr/bin/cage -s -- /opt/pili-pos/pili-pos
StateDirectory=pili-pos
StateDirectoryMode=0750
RuntimeDirectory=pili-pos
RuntimeDirectoryMode=0750
UMask=0027
```

```
ProtectSystem=strict
ProtectHome=no
InaccessiblePaths=-/home -/root
PrivateTmp=yes
NoNewPrivileges=yes
SupplementaryGroups=input video render
```

`ProtectHome=yes` masquerait aussi `/run/user`, où Cage crée son socket Wayland (répertoire de session du compte `pos`) : le kiosque ne démarrerait pas. `/home` et `/root` sont donc rendus inaccessibles explicitement ; le répertoire de `pos` est `/var/lib/pili-pos`.

L'opérateur n'a aucun moyen de s'élever en `root` (cf. §3.4 ci-dessous), donc aucun moyen de manipuler les permissions ou d'écrire ailleurs que dans `/var/lib/pili-pos/`.

Limite documentée honnêtement : un sandboxing syscall/noyau plus strict (`ProtectKernelTunables=`, `ProtectKernelModules=`, `ProtectControlGroups=`, `RestrictAddressFamilies=`, `SystemCallFilter=@system-service`) est une amélioration prévue mais **pas encore appliquée** à l'unité réelle : elle n'a jamais été testée contre le runtime WebKitGTK/Cage embarqué (risque réel de casser le rendu JIT de WebKitGTK sans un test d'exécution), et aucun boot réel n'a encore permis de le valider (cf. le rapport final Phase 4). Ne pas la déclarer comme active tant qu'elle n'a pas été testée.

Sur l'image d'admission uniquement (`--variant admission`), un second compte, `spf`, est créé dans le groupe `pos` - lecture seule de `pos.db` via `pos-db-ro` (§3.6), console `tty2`, mot de passe aléatoire généré et affiché une seule fois au premier démarrage (`MANUEL_INSTALLATION_FR.md` §3.4). Absent de l'image de production.

3.3 Image scellée

L'image kiosque livrée est **scellée**, avec les précisions suivantes (à ne pas sur-vendre) :

- Pas de serveur SSH, pas de Bluetooth, CUPS, Avahi, ModemManager : masqués (`systemctl mask`, symlinks vers `/dev/null`) à la construction de l'image.
- `tty1` est exclusivement occupé par `pili-pos.service` (`Conflicts=getty@tty1.service`) : **aucun** prompt de connexion n'y apparaît jamais derrière le kiosque.
- `tty2` et suivants gardent leur `getty/autovt` standard Debian - c'est le canal de maintenance (`Ctrl+Alt+F2`, login `root`, `MANUEL_INSTALLATION_FR.md` §4.1). Sur l'image d'admission uniquement, `getty@tty2.service` est explicitement activé pour le compte `spf` (voir §3.2). Ce n'est donc **pas** une image sans aucune console active : c'est une image où seul `tty1` (celui visible à l'écran) est verrouillé sur le kiosque.
- L'image est un Debian 13 "trixie" minimal construit par mkosi (paquets listés dans `deploy/provision/packages.txt` + `deploy/image/mkosi.conf`), pas du busybox : les utilitaires standard (`cat`, `cp`, `mv`, coreutils GNU) sont présents. `sqlite3` **est** présent (requis par `pili-pos` lui-même et par le wrapper `pos-db-ro`, §3.6). Un éditeur minimal (`vim-tiny`, fournissant `/usr/bin/vi`) est installé pour la maintenance de `pos.json` (§4.2 du manuel d'installation) ; `nano`/`less` restent absents.
- Cage Wayland compositor démarre directement sur pili-pos en plein écran, sans gestionnaire de fenêtres ni écran de connexion.

Assistance à distance (RustDesk). L'image n'exécute aucun service d'accès distant en permanence : RustDesk (version épinglée, installée dans `/opt/pili-pos/rustdesk/` par `provision.sh`) n'est lancé que sur demande, par `crates/pos-engine/src/commands/support.rs` :

- seul un Gérant connecté peut démarrer une session, et son PIN est redemandé à ce moment (`confirmed_operator`, accès `Admin`) ;
- une bannière est affichée sur chaque écran, écran de connexion compris, tant que la session dure : le personnel sait qu'une personne est connectée ;
- n'importe qui, connecté ou non, peut arrêter la session depuis la bannière ;
- la session se termine seule après 2 heures (`MAX_SESSION`), à la sortie du programme ou à l'arrêt de l'application ;
- une seule session à la fois ; le programme tourne avec un `HOME` privé sous le répertoire d'exécution du service : rien de ce qu'il écrit ne survit à un redémarrage ;

- le démarrage et l'arrêt sont journalisés (avec le rôle de la personne, sans nom ni INSZ).

Ce mécanisme ne donne pas d'accès à la base par lui-même : la protection de `pos.db` (triggers append-only, §2.1) s'applique à toute session.

Cf. `MANUEL_INSTALLATION_FR.md` pour les détails de construction d'image et `deploy/image/` (mkosi) pour la configuration source.

3.4 Pas d'élévation de privilèges

- Pas de `sudo` configuré sur l'image kiosque.
- L'utilisateur `pos` (et `spf` sur l'image d'admission) n'est dans aucun groupe sensible (`wheel`, `sudo`, `adm`…) ; `spf` est membre du groupe `pos` uniquement, pour la lecture seule de `pos.db`.
- L'opérateur (utilisateur du tactile) ne dispose même pas d'un compte shell ; l'écran tactile démarre pilipos en kiosk-mode. Seuls le distributeur (`root`, `tty2`) et, sur l'image d'admission, le contrôleur SPF (`spf`, `tty2`) ont un shell.

3.5 Boot sécurisé (BIOS / procédural)

- BIOS verrouillé par mot de passe (à configurer à l'installation, cf. `MANUEL_INSTALLATION_FR.md` §3.1 et `CHECKLIST_DEPLOIEMENT_KIOSK_FR.md`).
- Boot depuis le disque interne en usage normal ; **Secure Boot reste désactivé** (aucune chaîne de signature de l'UKI n'existe à ce jour - voir la limite documentée en §3.6).
- Cette couche est **procédurale** (dépend du technicien qui configure correctement le BIOS) : elle ne suffit plus, à elle seule, à protéger `pos.db` en cas de vol physique du terminal ou de disque - voir §3.6 pour le mécanisme qui prend réellement le relais.

3.6 Chiffrement LUKS2 de `/var` avec déverrouillage TPM2

Nouveau depuis l'image mkosi (`deploy/image/`) : la partition portant `/var` (donc `/var/lib/pili-pos/pos.db`) n'est **pas** en clair sur le disque. Au tout premier démarrage du terminal (jamais rejoué ensuite, `ConditionPathIsMountPoint=!/var`), `pili-provision.service` (`deploy/image/mkosi.extra/usr/local/lib/pil pos/pili-provision`) :

1. Formate la partition `pili-data` en LUKS2 avec une clé temporaire aléatoire.
2. Enrôle un jeton **TPM2** (`systemd-cryptenroll --tpm2-device=auto --tpm2-pcrs=7`, scellé au PCR 7 - état Secure Boot) pour un déverrouillage automatique et silencieux à chaque démarrage normal.
3. Enrôle une **clé de récupération** (`systemd-cryptenroll --recovery-key`), affichée **une seule fois** à l'écran à la fin du premier démarrage, avec le mot de passe `root` (et `spf` sur l'image d'admission) générés dans la foulée - à noter immédiatement dans le carnet du distributeur (`MANUEL_INSTALLATION_FR.md` §3.4). Ni ces mots de passe ni la clé de récupération ne sont conservés nulle part sur le disque après cet affichage unique.
4. Supprime la clé temporaire (`cryptsetup luksKillSlot`) : dès lors, seuls le TPM2 de **ce terminal précis** ou la clé de récupération permettent de déverrouiller `/var`.

Conséquence pratique : un vol du disque seul (ou du terminal entier, tant que l'attaquant ne peut pas aussi obtenir la clé de récupération consignée par le distributeur) ne donne accès ni à `pos.db`, ni aux sauvegardes, ni à `pos.json` - la partition racine (`/`, en lecture seule, contenant le binaire et le système) reste en clair mais ne contient aucune donnée fiscale.

Limite documentée honnêtement : le scellement au PCR 7 mesure l'état de la politique Secure Boot, mais Secure Boot lui-même reste **désactivé** sur ces images (aucune infrastructure de signature de l'UKI n'a été mise en place - voir `deploy/kiosk/README.md` §"Still to do"). Le scellement TPM2 protège donc contre l'extraction à froid du disque (vol du SSD seul) et détecte toute modification ultérieure des variables UEFI liées à Secure Boot, mais **pas** contre un attaquant qui altère le firmware/bootloader du terminal lui-même pendant que Secure Boot est désactivé - dans ce cas, la clé de récupération gardée par le distributeur reste la seule garantie de reprise en main, pas une protection contre l'accès. Aucun démarrage réel sur matériel physique avec TPM2

n'a validé ce flux de bout en bout au moment de la rédaction de ce document ; seul un test QEMU+swtpm partiel a été tenté (voir le rapport final Phase 4 et `scripts/qemu/run-kiosk.sh`).

Accès en lecture seule à `pos.db` une fois `/var` déverrouillé, sans jamais l'ouvrir en écriture : wrapper `pos-db-ro` (`/usr/local/bin/pos-db-ro`), disponible au compte `spf` (groupe `pos`) sur l'image d'admission - voir §3.2 et `FEUILLE_ACQUISITION_DONNEES_FR.md` §2.2.

4. Mécanismes au niveau cryptographique

4.1 Signature ECDSA secp256r1 (par le FDM)

Chaque événement fiscal porte la signature `digitalSignature` du FDM dans `fiscal_events.fdm_response_json`, conservée telle que le FDM l'a renvoyée, avec la réponse complète. Le FDM garde lui-même chaque événement signé et le transmet au SPF Finances. Une ligne `fiscal_events` modifiée a posteriori (par exemple via un accès physique) ne correspondrait donc plus à l'événement signé détenu par le FDM et le SPF Finances : la comparaison révèle la falsification.

Le POS ne détient pas la clé publique du FDM (le schéma GraphQL du FDM ne l'expose pas) et ne vérifie donc pas lui-même la signature ECDSA. Il contrôle en revanche, avant tout archivage, que chaque réponse correspond à la requête envoyée (`crates/pos-fiscal/src/response_check.rs`) :

- `posId`, `posFiscalTicketNo`, `posDateTime`, `deviceId` et `terminalId` identiques à la requête ;
- `eventOperation` et `eventLabel` conformes à la mutation appelée (label T en mode formation) ;
- `digitalSignature` présente et en base64 valide ;
- pour un événement N : `shortSignature` présente et égale à Hex(SHA1(Base64Decode(digitalSignature))), `verificationUrl` présente.

Une réponse qui ne respecte pas ces règles n'est jamais archivée : la vente est refusée, rien n'est imprimé, la commande reste ouverte et l'écran affiche « Réponse de la boîte noire incohérente ». Un compteur FDM qui ne progresse pas (même FDM) est consigné dans le journal technique.

4.2 Hash SHA-1 du binaire

Le hash SHA-1 du binaire `pili-pos` certifié est consigné dans le formulaire de demande de certification. Le contrôleur peut à tout moment vérifier sur le kiosque :

```
sha1sum /opt/pili-pos/pili-pos
```

Toute différence avec la valeur consignée prouverait que le binaire exécuté n'est pas celui certifié.

4.3 Hash du code source

Tag git signé du commit de la révision certifiée. Permet de reproduire le binaire à l'identique et de vérifier l'absence de modifications post-certification.

5. Sauvegardes

Voir `FEUILLE_ACQUISITION_DONNEES_FR.md` §1.2.

Politique : - Sauvegarde automatique après chaque Z-report, juste après son archivage (`signing::follow_up` pour un Z, qui appelle `commands::report::backup_database_impl`, via SQLite Online Backup API). Un échec de sauvegarde est journalisé et n'annule pas le Z. - Fichier daté à la seconde, **jamais écrasé**. - Stockage local. Aucune sauvegarde n'est envoyée hors de la caisse par défaut ; la seule copie distante prévue est l'export nocturne facultatif et chiffré du §5.1. - Rétention indéfinie (au moins 7 ans, exigence légale belge).

5.1 Sauvegarde chiffrée de bout en bout (export nocturne, facultatif)

Seulement si la section `export` de `pos.json` est renseignée (`EXPORT_NOCTURNE_FR.md`). Elle s'ajoute aux sauvegardes locales ci-dessus, qu'elle ne remplace pas ni ne modifie.

- **Chiffrement de bout en bout.** La copie de `pos.db` est chiffrée sur la caisse avec age (X25519) pour la clé publique du propriétaire de l'établissement et, en option, du producteur, avant d'être envoyée. Les clés privées ne sont jamais sur la caisse ni sur le serveur de réception, qui ne détient la sauvegarde que sous forme chiffrée et ne peut pas la lire (il reçoit aussi la copie statistique et le manifeste, non chiffrés).
- **Aucun texte clair conservé.** La copie intermédiaire de la base n'existe que le temps du chiffrement, dans un répertoire de travail vidé au démarrage et avant chaque préparation ; elle est supprimée aussi en cas d'échec. Seule la copie statistique (sans donnée directement identifiante, opérateurs sous pseudonyme) est conservée en clair, avec le fichier chiffré.
- **Intégrité.** Chaque fichier est envoyé avec son empreinte SHA-256, que le récepteur vérifie ; le manifeste, envoyé en dernier, répète les empreintes et les tailles et marque l'export complet.
- **Transport.** HTTPS vers le point de réception propre à l'installation ; jeton porteur dans un fichier séparé de `pos.json`, jamais journalisé.
- **Aucune écriture dans la base.** La préparation lit `pos.db` par une connexion en lecture seule, hors du verrou de signature : elle ne peut ni modifier `fiscal_events` ni retarder une vente ou un Z.
- **Stockage local** dans la mémoire interne, à côté de `pos.db` (protégée comme elle : permissions du service, chiffrement LUKS2 de `/var`, §3.6), jamais sur une carte SD.

6. Confidentialité (GDPR)

Voir `FEUILLE_ACQUISITION_DONNEES_FR.md` §6 et `PROFIL_ENTREPRISE_FR.md` §GDPR.

Données de l'équipe :

- INSZ stocké en clair (donnée personnelle obligatoire fiscalement).
- PIN stocké en hash bcrypt (jamais en clair).
- Aucun INSZ n'est écrit dans les journaux applicatifs.

Données de clients. Le binaire certifié contient l'intégration HubRise (désactivée par défaut, activée par installation : `COMMANDES_EN_LIGNE_FR.md` §2). Dès qu'elle est activée, le POS traite des données de clients, ce qu'il ne faisait pas sans elle :

- **Commandes en ligne.** Chaque commande reçue est conservée telle que HubRise l'envoie dans `orders.raw_hubrise_json`, avec les données du client (nom, téléphone, adresse de livraison, notes), et dans `hubrise_events.payload_json`. Le ticket client non fiscal imprime le nom, le téléphone et l'adresse. Aucune purge automatique de ces colonnes n'existe : elles suivent la conservation de la base.
- **Ventes de la caisse renvoyées à HubRise.** Quand l'intégration est activée, chaque vente de la caisse est mise en file dans `hubrise_outbox` puis envoyée à HubRise (lignes, totaux, paiements ; pas de données de client).
- **Ticket numérique par e-mail.** L'adresse saisie est conservée en clair dans `ticket_deliveries.recipient` le temps de l'envoi, puis effacée (envoi réussi, repli papier, ou au plus tard 72 h après) ; une forme masquée reste. Le ticket et l'adresse du destinataire sont transmis à Scaleway Transactional Email pour l'envoi.
- **Facture B2B** : hors binaire certifié v1.0.0 (aucune donnée de client B2B n'est donc traitée).

Jeton HubRise. `hubrise_connection.access_token` (jeton d'accès OAuth au compte HubRise de l'établissement) est stocké **en clair** dans `pos.db`. Il est donc lisible par le compte `spf` de l'image d'admission (lecture seule via `pos-db-ro`, §3.2) et présent dans chaque sauvegarde de la base. Il est protégé comme le reste de la base : permissions `0640` du groupe `pos`, chiffrement LUKS2 de `/var` (§3.6). Il est supprimé à la déconnexion de HubRise.

Export : par défaut, le POS n'exporte automatiquement aucune donnée fiscale ; les seuls flux sortants sont ceux ci-dessus (HubRise et Scaleway, selon la configuration de l'installation), la communication avec le FDM et, si la section `export` de `pos.json` est renseignée, l'export nocturne (§5.1) : une sauvegarde chiffrée de bout en bout, que le serveur de réception ne peut pas lire, et une copie statistique sans donnée directement identifiante (`EXPORT_NOCTURNE_FR.md`).

7. Audit trail croisé

Chaque événement fiscal est tracé à TROIS endroits indépendants :

1. **Côté POS** : table `fiscal_events`.
2. **Côté FDM** : journal interne du FDM (signe et stocke indépendamment).
3. **Côté SPF Finances** : remontée cloud quotidienne du FDM via le protocole `FPSFIN_API_PROTOCOL_FDM_FINCLOUD_1.`

Le contrôleur peut croiser ces trois sources pour repérer toute divergence. Une suppression locale de `fiscal_events` serait immédiatement visible par la non-correspondance avec le journal FDM.